March U: a test for unlinked memory faults

A.J.van de Goor
G.N.Gaydadjiev

Indexing terms: Memory tests, March tests, Memory faults, Fault model, Simple faults, Disturb fault

Abstract: Short and efficient memory tests is the goal of every test designer. To reduce the cost of production tests, often a simple test which covers most of the faults, e.g. all simple (unlinked) faults, is desirable to eliminate most defective parts; a more costly test can be used thereafter to eliminate the remainder of the bad parts. Such a test-cost efficient approach is used by most manufacturers. In addition, system power-on tests are not allowed a long test time while a high fault coverage is desirable. The authors propose a new realistic fault model (the disturb fault model), and a set of tests for unlinked faults. These tests have the property of covering all simple (unlinked) faults at a very reasonable test time compared with existing tests.

1 Introduction

Testing semiconductor memories is becoming a major cost factor in the production of modern memory chips; hence the selection of the most appropriate test, given a particular set of fault models, is becoming of increasing importance. In addition, for certain types of tests, such as power-on tests, a tradeoff between fault coverage and test time usually has to be made. The major reason for march tests to demand a large test time is due to the requirement to detect linked faults; this can be attributed to the many different ways faults can be linked. For the elimination of the majority of the bad parts during production, as well as for power-on tests, a short test time is required. Tests for simple (unlinked) faults have that property. However, to have a 'reasonable' fault coverage, a new fault model, the disturb fault model [10], has been introduced because of its widespread industrial use [9].

2 Functional faults in SRAMs

Depending on the way faults manifest themselves, faults can be classified as 'simple faults' (a simple fault is a fault which does not influence the behaviour of other faults), and 'linked faults', which do influence the behaviour of other faults such that masking may occur [5, 14]. In this paper the term 'fault type' denotes a particular class of faults, while 'fault subtype' denotes one of the ways a fault type can manifest itself; e.g. a SAF is a fault type, while the SA0 and SA1 faults are subtypes. The term 'fault' is used to denote either a fault type or a subtype.

IEE Proceedings online no. 19971147

Paper first received 30th July 1996 and in revised form 21st January 1997

The authors are with Delft University of Technology, Department of Electrical Engineering, Section Computer Architecture & Digital Technique, Mekelweg 4, 2628 CD Delft, The Netherlands

2.1 Simple faults

Simple faults are faults which are not linked; they can be classified as address decoder faults (AFs), and memory cell array faults (MCAFs) consisting of single cell faults, and MCAFs consisting of faults between memory cells [1]. For describing the latter two faults, the following notation is used:

< ... > denotes a particular fault; ' ... ' describes the fault. < S/F > denotes a fault in a single cell; S describes the value/operation sensitising the fault: $S \in \{0, 1, \uparrow, \downarrow, \updownarrow\}$ and F describes the value of the faulty cell. $\forall$ denotes any operation; $\forall \in \{\uparrow, \downarrow, \updownarrow\}$, where $\uparrow$ denotes an up, $\downarrow$ denotes a down transition write operation, and $\updownarrow$ denotes an $\uparrow$ or a $\downarrow$ transition write operation. A 'transition write operation' is a $w\bar{x}$ operation to a cell containing the value x, $x \in \{0, 1\}$.

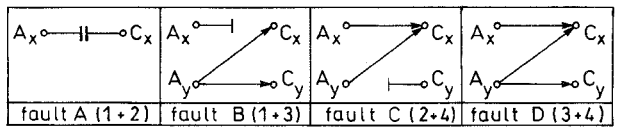

Fig. 1 *Combinations of address decoder faults*

2.1.1 Address decoder faults: An address decoder fault (AF) can only be present in the address decoder. Of this fault type four different subtypes exist [4]. A particular subtype cannot occur alone, but only in combination with at least one other subtype, see faults A, B, C and D in Fig. 1.

1. Fault 1: with a certain address no cell will be accessed
2. Fault 2: there is no address with which a particular cell can be accessed
3. Fault 3: with a certain address, multiple cells are accessed simultaneously
4. Fault 4: a certain cell can be accessed with multiple addresses

2.1.2 Single cell faults: Single cell faults are restricted to a single cell of the memory cell array. This class consists of the following fault types.

Stuck-at fault (SAF): The logic value of a stuck-at cell or line is always 0 (a SA0 fault) or always 1 (a SA1 fault). Notation for a SAx fault is < $\forall/x$ >, $x \in \{0, 1\}$. The fault type SAF has two subtypes: the SA0 and SA1 faults.

Stuck-open fault (SOF): An SOF in a memory cell means that the cell cannot be accessed; e.g. owing to an open word line [2, 3]. When the sense amplifier contains a latch, then upon a read operation the previous read value may be produced.

Transition fault (TF): A cell fails to undergo a $0 \rightarrow 1$ transition ($a < \uparrow /0 > TF$) and/or a $1 \rightarrow 0$ transition ($a < \downarrow /1 > TF$). The fault type TF has two subtypes: the $< \uparrow /0 >$ and the $< \downarrow /1 >$ TFs.

Data retention fault (DRF): A cell fails to retain its logic value after some period of time; this is caused in an SRAM by a broken (open) pull-up device [2, 3]. The fault type DRF has two subtypes: $< 1_T/ \downarrow >$ (a stored '1' will become a '0' after a time T) and $< 0_T/ \uparrow >$; where T denotes the amount of time required for the fault to develop. If both subtypes are present in a single cell, this cell behaves as if it contains an SOF [6].

2.1.3 Faults between memory cells:

A coupling fault (CF) involves two cells; one cell (cell j) sensitising the fault caused in another cell (cell i). Cell j is said to be the coupling cell, whereas cell i is said to be the coupled cell. Several types of CFs can be distinguished. The following notation for CFs is used: $< S_1, ..., S_{m-1}; F >$ denotes a fault involving m cells; $S_1, ..., S_{m-1}$ describe the conditions of the $m-1$ cells to sensitise the fault in cell m (denoted by F); $S_i \in \{0, 1, \uparrow, \downarrow, \updownarrow\}$ for $1 \leq i \leq m-1$. The fault '$\updownarrow$' denotes that the coupled cell changes from the value x to $\bar{x}$, $x \in \{0, 1\}$.

An *inversion coupling fault (CFin)* is defined as follows: an $\uparrow$ and/or a $\downarrow$ transition write operation in the coupling cell causes an inversion in the coupled cell. This results in the following two CFin subtypes: $< \uparrow; \downarrow >$ and $< \downarrow; \uparrow >$.

An *idempotent coupling fault (CFid)* is caused by an $\uparrow$ and/or a $\downarrow$ transition write operation in the coupling cell which forces a certain value (0 or 1) in the coupled cell. The following four CFid subtypes exist: $< \uparrow; \downarrow >$, $< \uparrow; \uparrow >$, $< \downarrow; \downarrow >$ and $< \downarrow; \uparrow >$.

In case of a *state coupling fault (CFst)* a coupled cell i is forced to a certain value x only if the coupling cell is in a given state y, i.e. has the value y [2, 3]. Four CFst subtypes exist: $< 0; 0 >$, $< 0; 1 >$, $< 1; 0 >$ and $< 1; 1 >$.

A *disturb coupling fault (CFdst)* is a fault whereby the coupled cell is disturbed (i.e. makes an $\uparrow$ or a $\downarrow$ transition) due to a ry or a wy operation ($y \in \{0, 1\}$) applied to the coupling cell [9, 10, 12]. Eight CFdst subtypes exist: $< r0; \uparrow >$, $< r0; \downarrow >$, $< r1; \uparrow >$, $< r1; \downarrow >$, $< w0; \uparrow >$, $< w0; \downarrow >$, $< w1; \uparrow >$ and $< w1; \downarrow >$. Emperical results have shown the effectiveness of this fault model [7, 9], which is due to the fact that also read operations as well as transition write and non-transition write operations can sensitise faults. Considering the way memories work, this is a much more realistic fault model than the CFid and CFin fault models which only assume faults sensitised by transition write operations.

Note that for SRAMs read disturb coupling faults are unlikely to occur; because after the precharge operation, a voltage difference between the bitlines BL and $\overline{BL}$ of typically on the order of 100mV is sufficient for the sense amplifier. Hence the BL or $\overline{BL}$ lines barely make a $\downarrow$ transition during a read operation. Therefore, for SRAMs, the CFdst fault model, which assumes disturbs by read as well as by write operations, can be simplified to disturb by write operations only: CFdst-w $< x; y >$; $x \in \{w0, w1\}$ and $y \in \{\uparrow, \downarrow\}$. The write-disturb, CFdst-w, fault type has four subtypes: $< w0; \uparrow >$, $< w0; \downarrow >$, $< w1; \uparrow >$ and $< w1; \downarrow >$. For DRAMs, due to the write-back operation, also read operations can cause disturb coupling faults; hence, the CFdst fault type applies.

2.1.4 Neighbourhood pattern sensitive faults:

A neighbourhood pattern sensitive fault (NPSF) [5] consists of single cell fault (a SAF or a TF) or a two-cell fault (a CF) whereby additional cells are involved; the single cell or two cell fault will only occur when the additional cells have an enabling value (i.e. are in a particular state), denoted by **E**. The following NPSFs are recognised:

- Static NPSF (SNPSF): $< E; \forall/x >$, whereby $x \in \{0, 1\}$
- Passive NPSF (PNPSF): $< E; \uparrow /0 >$ or $< E; \downarrow /1 >$
- Active NPSF (ANPSF): $< E, S; F >$, whereby S represents the sensitising condition of the two cell fault F.

The number of cells involved in a NPSF is denoted by k; which represents the size of the neighbourhood. Typical neighbourhoods are the Type-1 neighbourhood ($k = 5$) and the Type-2 neighbourhood ($k = 9$). The cell where the fault occurs is called the 'base cell'; while the group of cells, excluding the base cell is referred to as the 'deleted neighbourhood'.

2.2 Linked faults

A linked fault involves two or more simple faults. A CF is linked with another CF when both CFs have the same coupled cell (this may cause fault masking). When a TF is linked with a CF, and the TF is in the coupling cell, the CF may not be sentisable; for example, the TF $< \uparrow /0 >$ in cell j linked with the CFid $< \uparrow; \uparrow >$ whereby cell j is the coupling cell. When the TF is in the coupled cell, the TF may be masked.

Fig. 2 shows the general form of linked CFs; cell i is coupled to a cells j all with addresses lower than i, and b cells k all with addresses higher than i. Masking may occur in case of the following two CFids $< \uparrow; \uparrow >_{j1i}$ and $< \uparrow; \downarrow >_{j2i}$ because the first CF causes the cell to be set, which is thereafter reset by the second CF. The notion of linked faults has been introduced to establish more clearly the idea of simple (i.e. unlinked) faults. Linked faults will not be considered any further in this paper.

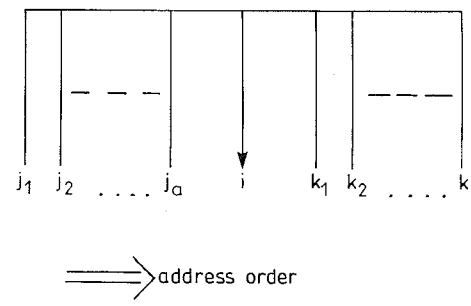

Fig. 2 *General form of linked CFs*

3 March tests

March tests consist of a family of tests which all have the same structure; they have proven to be superior in terms of test time and simplicity [1, 8, 15, 16].

Section 3.1 introduces the notation used to describe march tests. Similar to the notation used in mathemat-

ics, a good notation allows for a compact, concise description of the subject matter. Section 3.2 lists a set of well-known and commonly used march tests.

3.1 March notation

A march test consists of a sequence of march elements; a march element consists of a sequence of operations which are all applied to a given cell, before proceeding to the next cell. The way one proceeds to the next cell is determined by the address order which can be an increasing address order (increasing addresses from cell 0 to cell $n-1$), denoted by the '$\Uparrow$' symbol, or a decreasing address order, denoted by the '$\Downarrow$' symbol. (Any address sequence may be used for the $\Uparrow$ address order; e.g. from 0 to $n-1$, but also a pseudo-randomly determined address sequence; n denotes the total number of addresses). The $\Downarrow$ address order has to be the exact inverse of the $\Uparrow$ address order [5]. For some march elements the address order can be chosen arbitrarily, this will be indicated by the '$\Updownarrow$' symbol. An operation, applied to a cell, can be a '$w0$' (write '0'), a '$w1$', a '$r0$' (read '0'), or a '$r1$' operation.

A complete march test is delimited by the '{...}' bracket pair; while a march element is delimited by the '(...)' bracket pair.

The following march test {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$)} is the MATS+ test [13]. It consists of three march elements, M_0, M_1 and M_2. M_2 performs a $r1$ operation, followed by a $w0$ operation on a cell, after which these two operations are applied to the next cell.

3.2 List of march tests

The following march tests will be analysed because they are either well-known and frequently used and/or they have special properties in terms of fault coverage. The new march tests, proposed in the forthcoming Section, have been included in the list for ease of comparison. Note that the IFA tests are identical to other tests, or are other tests extended to cover DRFs.

MATS: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Updownarrow$ ($r1$)} [13]

MATS+: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$)} [13]

MATS++: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$,$w1$); $\Downarrow$ ($r1$, $w0$, $r0$)} [5]

March X: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$); $\Updownarrow$ ($r0$)} [5]

March Y: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$, $r1$); $\Downarrow$ ($r1$, $w0$, $r0$); $\Updownarrow$ ($r0$)} [5]

March C: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Uparrow$ ($r1$,$w0$); $\Updownarrow$ ($r0$); $\Downarrow$($r0$, $w1$); $\Downarrow$ ($r1$, $w0$); $\Updownarrow$ ($r0$)} [11]

March C–: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Uparrow$ ($r1$, $w0$); $\Downarrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$); $\Updownarrow$ ($r0$)} [5]

MOVI: {$\Downarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$, $r1$); $\Uparrow$ ($r1$, $w0$, $r0$); $\Downarrow$ ($r0$, $w1$, $r1$); $\Downarrow$ ($r1$, $w0$, $r0$)} [10]

IFA-6: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$, $r0$)} [3]

This test is identical to the MATS++ test.

IFA-9: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$); $\Uparrow$ ($r1$, $w0$); $\Downarrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$); Del; $\Updownarrow$ ($r0$, $w1$); Del; $\Updownarrow$ ($r1$)} [3]

This test is identical to the March C– test, extended to detect DRFs which are not linked with other faults.

IFA-13: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$, $r1$); $\Uparrow$ ($r1$, $w0$, $r0$); $\Downarrow$($r0$, $w1$, $r1$); $\Downarrow$ ($r1$, $w0$, $r0$); Del; $\Updownarrow$ ($r0$, $w1$); Del; $\Updownarrow$ ($r1$)} [3].

Note: If M_0 is changed into $\Downarrow$ ($w0$) then this test is identical to MOVI, extended to detect DRFs.

March U: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$, $r1$, $w0$); $\Uparrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$, $r0$, $w1$); $\Downarrow$ ($r1$, $w0$)}

March U–: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$,$w1$,$r1$,$w0$); $\Uparrow$ ($r0$, $w1$); $\Downarrow$ ($r1$, $w0$, $w1$); $\Downarrow$ ($r1$,$w0$)}

March UD: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$, $r1$, $w0$); Del; $\Uparrow$ ($r0$, $w1$); Del; $\Downarrow$ ($r1$, $w0$, $r0$, $w1$); $\Downarrow$ ($r1$, $w0$)}

March UD–: {$\Updownarrow$ ($w0$); $\Uparrow$ ($r0$, $w1$, $r1$, $w0$); Del; $\Uparrow$ ($r0$, $w1$); Del; $\Downarrow$ ($r1$, $w0$, $w1$); $\Downarrow$ ($r1$, $w0$)}

4 An analysis of unlinked address decoder faults

Depending on the memory technology, the effect of AFs on memory read operations can be according to the following cases:

Case A: When no cell will be accessed with a certain address (Fig. 1, fault A and B) the effect of a read operation can be that:

1. The cell behaviour is that of a SAF. This applies to DRAMs, and to SRAMs which use a single bitline per column.

2. The cell behaviour is that of a SOF. This applies to SRAMs which use differential sense amplifiers.

Case B: When multiple cells, whereby at least one cell contains a '0' and at least one other cell contains a '1' value, are read with a certain address (Fig. 1, fault D) the result of the read operation can be:

1. Deterministic. This will be the case for DRAMs; they produce the AND or the OR of the read values. This case also applies to SRAMs which use a single bitline per column.

2. Not deterministic (random). This applies to SRAMs which use a differential sense amplifier.

Table 1 shows the memory types for which the four combinations of the cases A and B apply; note that the combinations A.1 – B.2 and A.2 – B.1 do not occur in real memories.

Table 1: Memory types for which cases A and B apply

Read not connected cell	Read multiple cells	Memory type
A.1 SAF	B.1: deterministic	AFdr: DRAM (also SRAM using a single bitline)
A.1: SAF	B.2: random	AFtr: traditional AF model (non-existing)
A.2: SOF	B.1: deterministic	non-existing
A.2 SOF	B.2: random	AFsr: SRAM

For testing AFs for DRAMs case A.1 and case B.1 apply, this AF will therefore be called 'AFdr'. For testing SRAMs case A.2 and case B.2 apply, this AF will be called AFsr. The AFs used traditionally in literature, and until this section in this report, do not distinguish between SRAM and DRAM and consist of case A.1 and B.2. This type of traditional AF will be referred to as AFtr from here on. In summary, this means that the fault type AF consists of three subtypes: AFsr, AFdr, and the AFtr; whereby the fault subtype AFdr consists of the following subsubtypes (as shown below): AFdr-and, AFdr-or, and AFdr-ao.

In order for a march test to detect all unlinked address decoder faults, it has to satisfy the following conditions:

AFdr: Three cases can be distinguished, depending on the result of reading multiple cells which do not all have the same value.

AFdr: condition 1dr-and: The read result is the AND of the read values. To detect unlinked AFdrs of this subsubtype (which are called 'AFdr-and'), the march test has to contain the following two pairs of march elements [5]:

1. $\Updownarrow$ (..., $w0$); $\Updownarrow$ ($r0$, ...)
2. $\Updownarrow$ (..., $w1$); $\Updownarrow$ ($r1$, ..., $w0$)

AFdr: condition 1dr-or: The read result is the OR of the read values. In order to detect unlinked AFdrs of this subsubtype (which are called 'AFdr-or'), the march test has to contain the following two pairs of march elements [5]:

1. $\Updownarrow$ (..., $w0$); $\Updownarrow$ ($r0$, ..., $w1$)
2. $\Updownarrow$ (..., $w1$); $\Updownarrow$ ($r1$, ...)

AFdr: condition 1dr-ao: The read result is the AND or OR of the read values (the memory technology is not known to the test designer). The march test has to contain the following three march elements to detect the AFdrs of the subsubtype AFdr-ao:

$$\Updownarrow (..., wx); \Updownarrow (rx, ..., w\bar{x}); \Updownarrow (r\bar{x}, ..., wx)$$

AFtr: condition 1tr: any AFtr is detectable by a march test which contains the following two march elements [4]:

$$\Uparrow (rx, ..., w\bar{x}); \text{ and } \Downarrow (r\bar{x}, ..., wx)$$

AFsr: condition 1sr: any AFsr is detectable by a march test which satisfies condition 1tr and, additionally, contains a march element of the form (which may be one of the march elements of condition 1tr):

$$\Updownarrow (rx, ..., r\bar{x}, ...)$$

Note that the tests of Table 2 detect AFsrs if and only if they detect AFs and SOFs; e.g. the first test in the Table which detects AFsrs is MOVI.

From the above conditions one can determine the following hierarchy (1 is the most strong condition) in conditions a march test has to satisfy in order to detect unlinked AFs. If condition 1sr, for detecting AFsrs, is satisfied, all other AFs (AFtrs, AFdrs, AFdr-ands, AFdr-ors, and AFdr-aos) will be detected. Next are: condition 1sr for AFsrs; condition 1tr for AFtrs; condition 1dr-ao for AFdrs, when the memory technology is not known; condition 1dr-and or condition 1dr-or for AFdrs, when the memory technology is known.

5 March U: a test for all simple (i.e. unlinked) faults

This Section introduces a set of conditions a march test has to satisfy to detect all simple (i.e. unlinked) faults of Section 2.1 which result in March U.

5.1 Conditions for detecting simple faults

This section derives conditions march tests have to satisfy to detect all simple MCAFs of Section 2.1.2 and 2.1.3.

SOFs: condition 2: any SOF is detectable by a march test which contains the following march element [6]: (..., rx, ..., $r\bar{x}$, ...).

SAFs and TFs: condition 3: SAFs and TFs (even when linked with other faults) can be detected by a march test which contains the following two march elements (or a single march element containing both elements):

1. (..., $w0$, $r0$, ...) to detect SA1 faults and $< \downarrow /1 >$ TFs
2. (..., $w1$, $r1$, ...) to detect SA0 faults and $< \uparrow /0 >$ TFs

When SAFs and unlinked TFS are considered, condition 3 can be simplified to condition 3U.

Condition 3U: SAFs and TFs will be detected by a march test which contains the following operations (which can be distributed over march elements in an arbitrary way): wx, $w\bar{x}$, $r\bar{x}$, wx, rx.

DRFs: condition 4: any march test can be extended to detect DRFs [6]. The detection of each of the two DRF subtypes requires that a memory cell be brought into the corresponding logic states; thereafter, certain time must pass for the DRF to develop (for SRAMs, empirical results show that a wait time of 100 ms is enough [3]). If we are interested in detecting simple DRFs only, the delay elements can be placed between any two pairs of march elements as follows: $\Updownarrow$ (..., wx); Del; $\Updownarrow$ (rx, ..., $w\bar{x}$); Del; $\Updownarrow$ ($r\bar{x}$,...) with x = '0' or '1'.

CFs: condition 5: a march test which contains one of the two pairs of march elements of case A and a pair of march elements of case B can detect all simple CFs (i.e. all CFins, CFids, CFsts, and all CFdsts). Case A will detect all $< y; \bar{x} >$ and case B all $< y; x >$ CFs, with x the expected state of the coupled cell and $y \in \{r0, r1, w0, w1\}$ (This set contains all sensitising operations for CFs). Note that $\uparrow$ is the same as a $w1$ after the memory is in state '0'.

Condition 5 can be explained as follows: The operations (rx, ... $w\bar{x}$, $r\bar{x}$, ..., wx) in the first march element of case A.1 will sensitise the CFs, because that march element contains all sensitising operations; i.e. both states are entered (for CFsts), both transitions write operations take place (for CFins and CFids) and it contains all sensitising operations for CFdsts. Case A.1 will detect the fault, when the value of the fault effect is $\bar{x}$, by the 'rx' operation of the first march element when the coupled cell has a higher address than the coupling cell, and by the 'rx' operation of the second march element when the coupled cell has a lower address than the coupling cell. Case A.2 has similar capabilities as case A.1, except that the fault will be detected by the 'rx' operation of the first march element if the coupled cell has a lower address than the coupling cell, etc. Case B is required to sensitise and detect the CFs $< y; x >$.

case A: 1. $\Uparrow$ (rx, ..., $w\bar{x}$, $r\bar{x}$, ..., wx); $\Updownarrow$ (rx, ...)
2. $\Downarrow$ (rx, ..., $w\bar{x}$, $r\bar{x}$, ..., wx); $\Updownarrow$ (rx, ...)

case B: 1. $\Uparrow$ ($r\bar{x}$, ..., wx, rx, ..., $w\bar{x}$); $\Updownarrow$ ($r\bar{x}$, ...)
2. $\Downarrow$ ($r\bar{x}$, ..., wx, rx, ..., $w\bar{x}$); $\Updownarrow$ ($r\bar{x}$, ...)

In case of SRAM tests the simpler CFdst-w fault type can be used instead of the CFdst fault type. This allows condition 5 to be simplified to condition 5S (for SRAMs).

Condition 5S: any march test which contains one of the two pairs of march elements of case A and a pair of march elements of case B can detect all simple CFs (i.e. all CFins, CFids, CFsts, and CFdst-ws).

case A: 1. $\Uparrow$ (rx, ..., $w\bar{x}$, ..., wx); $\Updownarrow$ (rx, ...)
2. $\Downarrow$ (rx, ..., $w\bar{x}$, ..., wx); $\Updownarrow$ (rx, ...)

case B: 1. $\Uparrow$ ($r\bar{x}$, ..., wx, ..., $w\bar{x}$); $\Updownarrow$ ($r\bar{x}$, ...)
2. $\Downarrow$ ($r\bar{x}$, ..., wx, ..., $w\bar{x}$); $\Updownarrow$ ($r\bar{x}$, ...)

$$\{\ \underset{M0}{\Updownarrow(w0)}\ ;\ \underset{M1}{\Uparrow(r0,w1,r1,w0)}\ ;\ \underset{M2}{\Uparrow(r0,w1)}\ ;\ \underset{M3}{\Downarrow(r1,w0,r0,w1)}\ ;\ \underset{M4}{\Downarrow(r1,w0)}\ \}$$

Fig. 3 *March U*

$$\{\ \underset{M0}{\Updownarrow(w0)}\ ;\ \underset{M1}{\Uparrow(r0,w1,r1,w0)}\ ;\ \underset{M2}{\Uparrow(r0,w1)}\ ;\ \underset{M3}{\Downarrow(r1,w0,w1)}\ ;\ \underset{M4}{\Downarrow(r1,w0)}\ \}$$

Fig. 4 *March U–*

$$\{\ \underset{M0}{\Updownarrow(w0)}\ ;\ \underset{M1}{\Uparrow(r0,w1,r1,w0)}\ ;\ \underset{M2}{Del}\ ;\ \underset{M3}{\Uparrow(r0,w1)}\ ;\ \underset{M4}{Del}\ ;\ \underset{M5}{\Downarrow(r1,w0,r0,w1)}\ ;\ \underset{M6}{\Downarrow(r1,w0)}\ \}$$

Fig. 5 *March UD*

$$\{\ \underset{M0}{\Updownarrow(w0)}\ ;\ \underset{M1}{\Uparrow(r0,w1,r1,w0)}\ ;\ \underset{M2}{Del}\ ;\ \underset{M3}{\Uparrow(r0,w1)}\ ;\ \underset{M4}{Del}\ ;\ \underset{M5}{\Downarrow(r1,w0,w1)}\ ;\ \underset{M6}{\Downarrow(r1,w0)}\ \}$$

Fig. 6 *March UD–*

Table 2: Test length and fault coverage of simple faults

March test	Length	AF	SAF	TF	CFst	CFin	CFid	CFdst-w	CFdst	DRF	SOF
MATS	$4n$	+[†]	+	–	–	–	–	–	–	–	–
MATS+	$5n$	+	+	–	–	–	–	–	–	–	–
MATS++	$6n$	+	+	+	–	–	–	–	–	–	–
March X	$6n$	+	+	+	–	+	–	–	–	–	–
March C	$11n$	+	+	+	+	+	+	+	+	–	–
March C–	$10n$	+	+	+	+	+	+	+	+	–	–
MOVI	$13n$	+	+	+	+	+	–	+	+	–	+
IFA-6	$6n$	+	+	+	–	–	–	–	–	–	–
IFA-9	$12n + 2D$	+	+	+	+	+	+	–	–	+	–
IFA-13	$16n + 2D$	+	+	+	+	+	+	+	+	+	+
March U	$13n$	+	+	+	+	+	+	+	+	–	+
March U–	$12n$	+	+	+	+	+	+	+	–	–	+
March UD	$13n + 2D$	+	+	+	+	+	+	+	+	+	+
March UD–	$12n + 2D$	+	+	+	+	+	+	+	–	+	+

[†]Assuming memory technology is known

5.2 March U: a test for all simple faults

This Section shows March U and proofs its fault detection capabilities. Thereafter, tests derived from March U: March U– for testing SRAMs, March UD to additionally detect DRFs, and March UD– to additionally detect DRFs in SRAMs, are given.

Fig. 3 shows March U which satisfies the conditions of Sections 4 and 5.1, and therefore detects all simple (unlinked) faults of Section 2.1. M0 is required for initialisation; condition 1sr is satisfied by M1 or M3, together with M2 and M4; condition 2 is satisfied by M1; condition 3 by M1 and M3; and condition 5 by M1 and M2 (case A.1) and by M3 and M4 (case B.2). March U therefore detects all: AFs (i.e. all AFdrs, AFtrs, and AFsrs), SOFs, TFs (single or linked with other CFs), single CFins, single CFids, single CFsts and single CFdsts.

In case of SRAM tests, March U can be simplified to March U– (see Fig. 4). The conditions 1sr, 2, 3U and 5S are satisfied as follows: M1, M2 and M4 satisfy condition 1sr; condition 2 is satisfied by M1; condition 3U by M0, M1 and M2; and condition 5S by M1 and M2 (case A.1) and by M3 and M4 (case B.2).

Fig. 5 shows March UD, which detects DRFs, in addition to the faults detected by March U. March UD satisfies condition 4 via the march elements M1 through M5.

Fig. 6 shows March UD– (for SRAMs), it detects DRFs, in addition to the faults detected by March U–.

6 Conclusions

A new fault model, the disturb fault model, has been highlighted. It has been argued that it is more realistic than the commonly used inversion and idempotent coupling fault models; industrial practice [7, 10, 12] supports the effectiveness of this fault model. In addition, a more comprehensive set of fault models for address decoder faults has been introduced, and a systematic way of designing new march tests, based on the conditions they have to satisfy to detect faults of each fault model, has been proposed. This changes the design of march tests from an art to a science.

A new set of march tests, March U and tests derived from March U, are proposed; they have a short test length for the stated fault coverage. Their quality as compared with the existing tests is shown below.

Table 2 shows the fault coverage of the march tests of Section 3.2 for the simple faults of Section 2.1. The tests are listed in the left column of the Table; the corresponding test length in the column 'length', while the fault coverage is itemised for each of the simple faults in the remaining columns. The fault coverage information has been derived using the memory fault simulator MAP [5]. From inspecting the Table it is obvious that the longer tests have a higher fault coverage and that only few tests detect SOFs and DRFs. However, any test can be extended (modified) to detect those two fault types [6].

By inspecting Table 2 one can see that MOVI does not detect all CFids. This is because the CFid $< \downarrow; 1 >$

ji (the coupling cell '*j*' has a lower address) will not be detected. MOVI can be extended with the march element '$\Updownarrow$ ($r0$)' in order to detect this CFid subtype.

March C– does not detect SOFs. To detect those faults one of the march elements (rx, $w\bar{x}$) has to be modified into (rx, $w\bar{x}$, $r\bar{x}$); thereby increasing the test length to $11n$. Considering SRAMs, the shortest tests for all faults (except DRFs) of Table 2 are March C– (i.e. the $11n$ version) and March U– (which is $12n$). The advantages of March U are the following:

(a) If other tests have already detected AFs, the march element M4 (of March U, March U–, March UD and March UD–) can be reduced with one operation (i.e. the $\Downarrow$ ($r1$, $w0$) becomes $\Updownarrow$ ($r1$)) thereby reducing the test length by $1n$.

(b) March tests usually do not have the property of being able to detect NPSFs. However, March U (and all derived tests) additionally detect NPSFs with the constraint that all cells in the neighbourhood have the same value. The NPSFs are detected as follows:

— ANPSFs: March elements M1 and M3 sensitise CFs; therefore when applied to a deleted neighbourhood cell, a CF is sensitised in the base cell. The fault is detected by the 'rx' operation of M1, M2, M3 or M4, e.g. for a Type-1 neighbourhood the following ANPSFs will be detected $< E, \uparrow; 1 >$, $< E, \downarrow; 1 >$, $< E, \uparrow; 0 >$ and $< E, \downarrow; 0 >$ for the case the CF is a CFid and the value of E = '0000' or '1111'. Similarly the ANPSFs for the CFs: CFin and CFdst are detected; provided that the value of E is '0000' or '1111'; which are the most realistic values in practice. In addition, all ANPSFs will be detected, regardless of the type and the size of the neighbourhood.

— PNPSFs: the deleted neighbourhood values of E = '0000' or E = '1111' (assuming a Type-1 neighbourhood) enable the TF in the base cell which is detected by M1 or M3.

— SNPSFs: those are sensitised and detected similar to PNPSFs.

— Concluding, March U (and the tests derived from March U) detect all NPSFs assuming a homogeneous neighbourhood (i.e. all deleted neighbourhood cells have the same value), which is very realistic for NPSFs. In addition they have this capability regardless of the size and the type of the neighbourhood.

In conclusion, March U– has the same test length as March C– (when extended to detect SOFs), or a shorter test length when AFs do not have to be detected; while March U (and tests derived from March U) are able to detect realistic NPSFs (i.e. NPSFs with a homogeneous neighbourhood value); for any neighbourhood type and size.

7 References

1 ABADIR, M.S., and REGHBATI, J.K.: 'Functional testing of semiconductor random access memories', *ACM Comput. Surv.*, 1983, **15**, (3), pp. 175–198

2 DEKKER, R., BEENKER, F., and THIJSSEN, L.: 'Fault modeling and test algorithm development for static random access memories'. Proceedings of IEEE international *Test* conference, Washington, DC, 1988, pp. 343–352

3 DEKKER, R., BEENKER, F., and THIJSSEN, L.: 'A realistic fault model and test algorithms for static random access memories', *IEEE Trans. Comp. (USA)*, 1990, **C–9**, (6), pp. 567–572

4 VAN DE GOOR, A.J., and VERRUIJT, C.A.: 'An overview of deterministic functional RAM chip testing', *ACM Comput. Surv.*, 1990, **22**, (1), pp. 5–33

5 VAN DE GOOR, A.J.: 'Testing semiconductor memories, theory and practice' (John Wiley & Sons, Chichester, UK, 1991)

6 VAN DE GOOR, A.J.: 'Using march tests to test SRAMs', *IEEE Des. Test Comp.*, March 1993, pp. 8–14

7 VAN DE GOOR, A.J.: Private communication with several memory manufacturers (Digital, Intel and Samsung), 1996

8 VAN DE GOOR, A.J., and SMIT, B.: 'Generating march tests automatically'. Proceedings international *Test* conference, 1994, pp. 870–878

9 VAN DE GOOR, A.J., GAYDADJIEV, G.N., YARMOLIK, V.N., and MIKITJUK, V.G.: 'March LR: a test for realistic linked faults'. Proceedings of 14th IEEE *VLSI Test* symposium, 1996, pp. 272–280

10 DE JONGE, J.H., and SMEULDERS, A.J.: 'Moving inversions test pattern is thorough, yet speedy', *Computer Des. (USA)*, May 1976, pp. 169–173

11 MARINESCU, M.: 'Simple and efficient algorithms for functional RAM testing'. Proceedings of IEEE international *Test* conference, 1982, pp. 236–239

12 NADEAU-DOSTIE, B., SILBERT, A., and AGARWAL, V.K.: 'Serial interfacing for embedded memories', *IEEE Des. Test*, 1990, **7**, (2), pp. 52–63

13 NAIR, R.: 'Comments on an optimal algorithm for testing stuck-at faults in random access memories', *IEEE Trans. Comp. (USA)*, 1979, **C–28**, (3), pp. 258–261

14 PAPACHRISTOU, C.A., and SAGHAL, N.B.: 'An improved method for detecting functional faults in random-access memories', *IEEE Trans. Comp. (USA)*, 1985, **C–34**, (2), pp. 110–116

15 SUK, D.S., and REDDY, S.M.: 'A march test for functional faults in semiconductor random-access memories', *IEEE Trans. Comp. (USA)*, 1981, **C–30**, (12), pp. 982–985

16 VEENSTRA, P.K., BEENKER, F.P.M., and KOOMEN, J.J.M.: 'Testing of random access memories: theory and practice', *IEE Proc. G*, 1988, **135**, (1), pp. 24–28